Introduction to Cybersecurity
CCY2001

Group Project Compendium

60 Project Ideas for Undergraduate Computer Science Students

Each project is designed for a team of 4 students

CCIT – Semester 4
Academic Year 2025 – 2026

Introduction

This compendium presents 60 cybersecurity project ideas designed for undergraduate computer science students enrolled in an Introduction to Cybersecurity course (CCY2001). All projects assume limited prior cybersecurity knowledge and only basic Python programming skills.

Each project is intended for a team of 4 students working collaboratively over a typical academic semester (8–14 weeks). Projects are organized into 8 thematic categories covering the major domains of cybersecurity, from foundational cryptography and network security to emerging challenges in IoT and AI security.

For every project, this document provides:

- A brief overview explaining the project's purpose and what students will learn.
- A step-by-step procedural guide covering the main implementation phases.
- A prerequisites list identifying required Python knowledge, libraries, and tools.
- A difficulty rating: Beginner, Intermediate, or Advanced.

Difficulty Key

Symbol	Level	Description
★	Beginner	Achievable with basic Python loops, strings, and functions. No external security tools required. Ideal starter projects.
★ ★	Intermediate	Requires 1–3 external Python libraries, a basic understanding of networking or databases, and some system setup.
★ ★ ★	Advanced	Demands integration of multiple technologies, deeper conceptual understanding, and a properly configured lab environment.

Category 1: Cryptography & Encoding

Project 1: Classical Cipher Toolkit ★ Beginner

Overview

Students build a command-line Python application that implements classical ciphers — Caesar, Vigenère, Atbash, and Rail Fence. The tool supports both encryption and decryption, and includes a basic frequency-analysis module so students can crack ciphertexts without the key. This project introduces the foundational concepts of confidentiality and demonstrates why historical ciphers are no longer secure.

Main Steps & Procedures

1. Research the history and mechanics of each cipher (Caesar, Vigenère, Atbash, Rail Fence).
2. Implement an encryption function for each cipher in Python.
3. Implement the corresponding decryption function.
4. Add a frequency-analysis module that suggests the most likely plaintext for Caesar cipher.
5. Build a menu-driven CLI interface for user interaction.
6. Test with known plaintext/ciphertext pairs to validate correctness.
7. Prepare a demonstration showing how easily classical ciphers are broken.

Prerequisites

- Python strings, loops, and functions
- Basic understanding of ASCII and character encoding
- No external libraries required
- Understanding of modular arithmetic (basic)

Project 2: File Encryption & Decryption Tool ★★ Intermediate

Overview

Students build a practical file encryption utility using Python's cryptography library, implementing AES-256 in CBC mode. The tool encrypts and decrypts files of any type, generates random IVs, derives keys from passwords using PBKDF2, and includes file integrity verification via HMAC. This project bridges classical theory with modern cryptographic practice.

Main Steps & Procedures

1. Study symmetric encryption concepts: key, IV, block modes (CBC vs ECB).
2. Install and explore the Python cryptography library.
3. Implement password-based key derivation using PBKDF2-HMAC-SHA256.
4. Build the file encryption function using AES-256-CBC with a random IV.
5. Build the decryption function that recovers the original file.
6. Add HMAC-SHA256 for file integrity verification.
7. Test with text files, images, and PDFs; confirm identical byte output after decryption.

Prerequisites

- Python file I/O and binary mode
- pip package installation
- Basic understanding of what encryption is
- Python cryptography library (pip install cryptography)

Project 3: Password Strength Analyzer & Secure Generator ★ Beginner

Overview

Students develop a two-part Python tool: a password analyzer that scores passwords based on NIST guidelines (length, character variety, presence in common-password lists), and a cryptographically

secure password generator. The project teaches authentication principles, brute-force attacks, and why randomness matters in security.

Main Steps & Procedures

1. Compile a list of the top 1,000 most common passwords (freely available online).
2. Implement scoring rules: length, uppercase, digits, symbols, dictionary match.
3. Build a visual strength meter with color-coded feedback (weak / moderate / strong).
4. Implement a secure password generator using Python's secrets module.
5. Add a passphrase mode (random words from a wordlist).
6. Demonstrate time-to-crack estimates for different password patterns.
7. Package as a reusable Python module and write a usage guide.

Prerequisites

- Python string methods and conditionals
- Python secrets and string modules
- Basic file I/O (for loading wordlists)
- No external libraries required

Project 4: Hash Function Explorer & Password Cracker ★★ Intermediate

Overview

Students explore cryptographic hash functions (MD5, SHA-1, SHA-256) by building a Python tool that demonstrates hashing, collision resistance, and the avalanche effect. They then implement a simple dictionary and rainbow-table attack to crack unsalted MD5 hashes, and compare the effect of salting on crack success rate. This illustrates why secure password storage uses salted hashing.

Main Steps & Procedures

1. Study and document the properties of MD5, SHA-1, and SHA-256.
2. Write code to demonstrate the avalanche effect by flipping one bit in input.
3. Build a hash generator that accepts user input and outputs all three hashes.
4. Implement a dictionary attack against a list of unsalted MD5 hashes.
5. Build a simple rainbow table and measure lookup speed vs. dictionary attack.
6. Implement salted SHA-256 hashing and show the rainbow table fails.
7. Compare crypt cost factor and its impact on brute-force resistance.

Prerequisites

- Python hashlib module (standard library)
- Python file I/O for wordlists
- Basic understanding of loops and dictionaries
- No external libraries required

Project 5: Steganography Tool: Hiding Data in Images ★★ Intermediate

Overview

Students implement an LSB (Least Significant Bit) steganography tool in Python that hides text or file data inside PNG images without visually altering them. They also build a detection module that uses statistical analysis to identify images likely containing hidden data. This introduces concepts of data hiding, covert channels, and digital forensics.

Main Steps & Procedures

1. Study the PNG file format and the concept of LSB encoding.

Prerequisites

- Python Pillow library (pip install Pillow)

2. Implement a function to encode a text message into the LSBs of RGB pixels.
3. Implement the corresponding decoder to extract the hidden message.
4. Extend to support hiding small binary files, not just text.
5. Build a steganalysis module using chi-square statistical analysis.
6. Test with various image types and measure visual quality degradation (PSNR).
7. Document the limitations: capacity, detectability, fragility to compression.

- Basic understanding of binary and bitwise operations
- Understanding of RGB color model
- Basic Python loops and list comprehensions

Project 6: Digital Signature Simulator ★★ Intermediate

Overview

Students build a Python tool that simulates digital signatures using RSA. The tool generates key pairs, signs documents, verifies signatures, and demonstrates what happens when a document is tampered with after signing. This teaches non-repudiation, integrity, and the role of public-key cryptography in authentication.

Main Steps & Procedures

1. Study the concept of asymmetric cryptography and how RSA works at a high level.
2. Use the cryptography library to generate an RSA-2048 key pair.
3. Implement a document signing function using RSA-PSS with SHA-256.
4. Implement signature verification and display pass/fail clearly.
5. Simulate a tamper attack: modify one byte of the signed document and re-verify.
6. Implement a simple PKI-style certificate with signer name and expiry.
7. Demonstrate the difference between signing and encrypting.

Prerequisites

- Python cryptography library
- Basic understanding of public/private key concepts
- Python file I/O
- Understanding of hash functions (Project 4 helpful but not required)

Project 7: Secure Messaging App (End-to-End Encryption Simulation) ★★★ Advanced

Overview

Students build a basic two-party secure messaging simulation using Python sockets and hybrid encryption (RSA for key exchange, AES for message encryption). One instance acts as server, one as client. They implement a simple key exchange handshake and encrypted message channel, demonstrating confidentiality and forward secrecy concepts.

Main Steps & Procedures

1. Set up a basic Python TCP client-server using the socket module.
2. Implement RSA key pair generation on both client and server at startup.
3. Build the key exchange handshake: exchange RSA public keys, then send AES session key encrypted with RSA.

Prerequisites

- Python socket module (basic)
- Python cryptography or PyCryptodome library
- Basic understanding of TCP/IP
- Completion of Project 2 (AES) and Project 6 (RSA) recommended

4. Encrypt all subsequent messages with AES-128-CBC using the shared session key.
5. Add message integrity: append HMAC to each message.
6. Simulate a man-in-the-middle attack and discuss how certificates prevent it.
7. Test the full conversation flow and document the protocol design.

Project 8: Blockchain Mini-Ledger ★★★ Advanced

Overview

Students implement a simplified blockchain in Python to understand how cryptographic hashing, proof-of-work, and chain integrity work together. The ledger stores simple transactions, and students demonstrate how tampering with a block invalidates the entire chain. This project demystifies blockchain fundamentals without requiring any prior knowledge of cryptocurrency.

Main Steps & Procedures

1. Define the Block data structure: index, timestamp, data, previous hash, nonce.
2. Implement SHA-256 hashing of the block's contents.
3. Implement a proof-of-work function (find nonce so hash starts with N zeros).
4. Build the chain by linking each block's hash to the next block's header.
5. Implement a chain validation function that detects any tampering.
6. Demonstrate a tamper attack: modify a past transaction and show the chain breaks.
7. Visualize the chain in a simple text-based diagram.

Prerequisites

- Python hashlib and json modules
- Python classes and object-oriented basics
- Understanding of hash functions (Project 4 helpful)
- Basic Python datetime module

Category 2: Network Security

Project 9: Port Scanner & Service Finger printer ★★ Intermediate

Overview

Students build a TCP port scanner in Python that probes a target host for open ports, identifies running services by grabbing banners, and generates a structured report. All scanning is restricted to localhost or an isolated lab VM. This teaches network reconnaissance fundamentals — the first step in any penetration test — and helps students understand how services expose themselves on a network.

Main Steps & Procedures

1. Set up an isolated lab environment (localhost or a dedicated VM).
2. Implement a single-threaded TCP connect scan using Python's socket module.
3. Add multi-threading to parallelize the scan and measure speed improvement.
4. Implement banner grabbing: send a probe and read the first 1024 bytes of response.
5. Map common ports to service names using a built-in lookup table.
6. Generate a formatted scan report (text or CSV).
7. Discuss ethical and legal implications: get written permission before scanning any real host.

Prerequisites

- Python socket module
- Python threading module (basic)
- Understanding of TCP/IP and port numbers
- A safe lab environment (localhost or controlled VM)

Project 10: Network Packet Sniffer & Analyzer ★★★ Advanced

Overview

Students build a basic packet sniffer using Python's Scapy or raw sockets that captures live traffic on a local interface. They parse Ethernet, IP, TCP/UDP headers, and display key fields in real time. The project demonstrates how unencrypted protocols leak sensitive data and why encryption matters in transit.

Main Steps & Procedures

1. Study the Ethernet, IP, and TCP/UDP header formats.
2. Set up Scapy in a controlled lab environment.
3. Write a packet capture loop that reads frames from a network interface.
4. Implement parsers to extract and display source/destination IP, ports, and protocol.
5. Filter by protocol (show only HTTP, DNS, or ICMP traffic).
6. Demonstrate that HTTP traffic reveals plaintext usernames/passwords.
7. Compare with HTTPS traffic to show the effect of encryption.

Prerequisites

- Python Scapy library
- Administrator/root access on the test machine
- Basic understanding of the OSI model
- Isolated network lab (never sniff on public networks)

Project 11: HTTP Traffic Log Analyzer ★ Beginner

Overview

Students write a Python script that parses Apache or Nginx web server access logs to detect suspicious activity: brute-force login attempts, directory traversal, SQL injection patterns in URLs, and

port scanning footprints. The output is a color-coded security report. This introduces blue-team analysis skills without requiring any live network access.

Main Steps & Procedures

1. Obtain a sample Apache/Nginx log file (freely available online or generate one).
2. Write a parser that reads each log entry and extracts IP, timestamp, URL, and status code.
3. Implement a detector for brute-force: same IP with 10+ 401/403 responses in a minute.
4. Detect directory traversal attempts: URLs containing '../' sequences.
5. Flag SQL injection indicators: keywords like UNION, SELECT, DROP in URL parameters.
6. Summarize top offending IPs and recommended firewall rules.
7. Output findings as a formatted text report with severity labels.

Prerequisites

- Python file I/O and string methods
- Python re (regular expressions) module
- Python collections module (Counter, defaultdict)
- Sample log files (no live server required)

Project 12: DNS Enumeration & Spoofing Demo ★★ Intermediate

Overview

Students build a DNS enumeration tool that queries DNS records (A, MX, NS, TXT, CNAME) for a given domain, and then simulate a DNS cache poisoning attack in an isolated lab environment using Python and Scapy. This project demonstrates how DNS works and why DNSSEC was introduced.

Main Steps & Procedures

1. Study DNS record types and the DNS resolution process.
2. Use Python's dnspython library to enumerate all record types for a test domain.
3. Build a subdomain brute-force tool using a wordlist.
4. Set up an isolated DNS lab (two VMs: client and mock DNS server).
5. Simulate DNS cache poisoning by injecting a forged response packet.
6. Demonstrate how the client is redirected to a wrong IP.
7. Discuss DNSSEC as a defense and how it prevents spoofing.

Prerequisites

- Python dnspython and Scapy libraries
- Basic understanding of DNS and UDP
- Two VMs or containers in an isolated network
- Python socket module

Project 13: Firewall Rule Simulator ★★ Intermediate

Overview

Students build a software firewall simulator in Python that processes a rule set (allow/deny by IP, port, and protocol) and evaluates simulated network packets against the rules. They test the system with various packet scenarios, experiment with rule ordering, and learn about stateless vs. stateful filtering concepts.

Main Steps & Procedures

1. Define a JSON-based rule format: source IP, destination IP, port, protocol, action.
2. Implement a packet matching engine that evaluates packets against rules in order.
3. Add support for wildcard rules (any IP, any port).

Prerequisites

- Python JSON and CSV modules
- Basic networking (IP addresses, ports, protocols)
- Python classes (basic OOP)
- No external libraries required

4. Load packet scenarios from a CSV file and process each through the firewall.
5. Implement a logging module that records allowed and blocked packets.
6. Demonstrate rule-order dependency: show how changing order changes outcomes.
7. Add a default-deny policy and test its effect on unknown traffic.

Project 14: VPN Concept Demonstrator (TLS Tunnel Simulation) ★★★ Advanced

Overview

Students build a simple TLS-encrypted tunnel between two Python processes using Python's ssl module, simulating a VPN concept. One process acts as a TLS server (VPN gateway), the other as a client. All data sent through the tunnel is encrypted. Students generate self-signed certificates and explore certificate validation.

Main Steps & Procedures

1. Generate self-signed X.509 certificates using Python's cryptography library.
2. Build a basic TCP server and wrap it with Python's ssl module.
3. Build the TLS client and establish a secure connection.
4. Demonstrate that data captured mid-stream (with a simulated sniffer) is unreadable.
5. Implement certificate pinning on the client side.
6. Simulate a self-signed certificate warning and explain certificate trust chains.
7. Document how a real VPN extends this concept to route all traffic.

Prerequisites

- Python ssl and socket modules
- Python cryptography library
- Basic TCP/IP and TLS concepts
- OpenSSL or cryptography library for cert generation

Project 15: Wi-Fi Network Security Auditor (Passive) ★★ Intermediate

Overview

Students build a passive Wi-Fi auditing tool using Python that reads a pre-captured PCAP file of Wi-Fi probe requests and beacon frames, identifies nearby SSIDs, their security protocols (WEP/WPA/WPA2/WPA3), and any devices broadcasting probe requests. The project explores wireless security protocols without transmitting any packets.

Main Steps & Procedures

1. Study the 802.11 frame format and Wi-Fi beacon/probe frame structure.
2. Use Scapy to parse a provided PCAP file of Wi-Fi frames.
3. Extract SSIDs, BSSIDs, and security capability fields from beacon frames.
4. Classify each network by security protocol (WEP, WPA, WPA2, WPA3, Open).
5. Extract device MAC addresses from probe requests.
6. Generate a risk report flagging WEP and open networks.
7. Discuss WPA2 KRACK and why WPA3 was introduced.

Prerequisites

- Python Scapy library
- A provided PCAP file (no live capture required)
- Basic understanding of Wi-Fi and 802.11
- Python pandas for report generation (optional)

Project 16: Network Intrusion Detection System (Rule-Based) ★★★ Advanced

Overview

Students build a simplified rule-based NIDS in Python, similar in concept to Snort. The system reads network traffic from a PCAP file, matches packets against a signature database (port scan, SYN flood, known malicious payloads), and raises alerts. This introduces how enterprise IDS/IPS tools work and how intrusion signatures are written.

Main Steps & Procedures

1. Study how Snort rules are structured: rule header + options.
2. Define a simplified JSON-based signature format for the student IDS.
3. Parse a PCAP file with Scapy and feed packets to the detection engine.
4. Implement detection rules for: port scan, SYN flood, ICMP flood, HTTP directory traversal.
5. Build an alert generator with severity, timestamp, and source IP.
6. Add a whitelist to suppress false positives.
7. Test the IDS against a set of malicious PCAP samples (available from public datasets).

Prerequisites

- Python Scapy library
- Python JSON module
- Basic TCP/IP and packet structure knowledge
- Public PCAP datasets (e.g., CICIDS, PCAPR)

Category 3: Web Security

Project 17: SQL Injection Exploit & Defense Lab ★★ Intermediate

Overview

Students build a deliberately vulnerable login form using Python Flask and SQLite, exploit it using multiple SQL injection techniques (in-band, boolean-based blind, UNION-based), then harden the application using parameterized queries and ORM best practices. This is the most impactful introductory web security project — seeing the attack work makes the defense unforgettable.

Main Steps & Procedures

1. Build a simple Flask login form connected to a SQLite database.
2. Leave the query vulnerable: use string concatenation to build SQL queries.
3. Demonstrate a classic authentication bypass using ' OR '1'='1.
4. Perform a UNION-based injection to extract the users table.
5. Implement a boolean-based blind injection to exfiltrate data character by character.
6. Refactor to use parameterized queries (sqlite3 ? placeholders) and show all attacks fail.
7. Add input validation and error suppression as additional hardening layers.

Prerequisites

- Python Flask (basic routes and templates)
- Python sqlite3 module
- Basic HTML forms
- Understanding of SQL SELECT statements

Project 18: Cross-Site Scripting (XSS) Attack & Patch ★★ Intermediate

Overview

Students build a simple Flask message board with a comment submission feature, demonstrate three types of XSS attacks (reflected, stored, and DOM-based), and then patch each vulnerability using output encoding, Content Security Policy headers, and the HTMLParser sanitizer. Students present both the working attacks and the patched versions side by side.

Main Steps & Procedures

1. Build a Flask message board that stores and displays user comments.
2. Inject a stored XSS payload (<script>alert('XSS')</script>) and observe execution.
3. Demonstrate a reflected XSS via a search URL parameter.
4. Demonstrate DOM-based XSS by injecting via URL fragment processed by JavaScript.
5. Patch stored and reflected XSS using Jinja2 autoescaping and html.escape().
6. Add a Content-Security-Policy header to prevent inline script execution.
7. Document each attack type, its impact (cookie theft, keylogging), and the corresponding fix.

Prerequisites

- Python Flask and Jinja2 templates
- Basic JavaScript (enough to read a script tag)
- Basic HTML
- Python html module (standard library)

Project 19: CSRF Attack Demonstrator & Token Defense ★★ Intermediate

Overview

Students build two Flask applications: a vulnerable banking app (transfer funds form) and a malicious attacker site. They demonstrate a CSRF attack where visiting the attacker's page silently submits a fund-transfer request to the bank app. They then implement CSRF token protection using Flask-WTF or a custom token system.

Main Steps & Procedures

1. Build a simple Flask app simulating a bank account with a money-transfer form.
2. Build a second malicious web page that auto-submits the transfer form via HTML.
3. Demonstrate that without CSRF protection, the transfer executes silently.
4. Implement CSRF tokens: generate a random token per session, embed in forms, validate on POST.
5. Show that the attack fails once tokens are in place.
6. Add the SameSite=Strict cookie attribute and explain its effect.
7. Discuss real-world CSRF incidents and the role of CORS headers.

Prerequisites

- Python Flask and sessions
- Python secrets module
- Basic HTML forms and HTTP POST/GET
- Flask-WTF (optional) or Python secrets for token generation

Project 20: Web Vulnerability Scanner (Lightweight) ★★★ Advanced

Overview

Students build an automated web vulnerability scanner in Python using the requests library. The scanner crawls a target web application, tests for reflected XSS, SQL injection indicators, open redirects, and missing security headers. All testing is limited to a locally hosted vulnerable app (DVWA or a custom lab target).

Main Steps & Procedures

1. Set up a local DVWA (Damn Vulnerable Web Application) or a custom Flask test target.
2. Build a web crawler using requests and BeautifulSoup to discover all pages and forms.
3. Implement a reflected XSS tester: inject a known payload and check if it appears in the response.
4. Implement a SQL injection tester: inject ' and check for database error messages.
5. Check HTTP response headers for missing security headers (CSP, X-Frame-Options, HSTS).
6. Test for open redirects in URL parameters.
7. Generate a structured vulnerability report with severity ratings and remediation advice.

Prerequisites

- Python requests library
- Python BeautifulSoup (bs4) for HTML parsing
- Basic understanding of HTTP requests and responses
- A locally hosted vulnerable target (DVWA or custom app)

Project 21: Broken Authentication: Session Hijacking Lab ★★★ Advanced

Overview

Students build a Flask application with a flawed session management system (predictable session tokens, no expiry, no invalidation on logout) and simulate session hijacking attacks. They then implement proper session management using secrets-based tokens, server-side session storage, and secure cookie attributes.

Main Steps & Procedures

1. Build a Flask app with login that generates a predictable session token (e.g., sequential integers).

Prerequisites

- Python Flask and sessions
- Python secrets module

2. Demonstrate session token prediction: enumerate tokens to hijack another user's session.
3. Show that logout does not invalidate the server-side session — hijacking persists.
4. Implement cryptographically random session tokens using Python's secrets module.
5. Add server-side session tracking with expiry and mandatory invalidation on logout.
6. Set Secure, HttpOnly, and SameSite cookie flags.
7. Demonstrate that each attack fails after hardening and explain why.

- Understanding of HTTP cookies and sessions
- Basic Flask routing and templates

Project 22: Directory Traversal & File Inclusion Attack Lab ★★ Intermediate

Overview

Students build a simple Flask file-serving app that naively reads files from the server based on user input, then exploit it using directory traversal (../../../etc/passwd) and local file inclusion techniques. They patch the app using path canonicalization and a strict allowlist, illustrating input validation principles.

Main Steps & Procedures

1. Build a Flask route that reads a file from a static directory based on a filename URL parameter.
2. Demonstrate directory traversal: request ../../../../etc/passwd.
3. Demonstrate that the app leaks its own source code via traversal.
4. Patch using os.path.realpath() and verify the file is inside the allowed directory.
5. Implement an allowlist of permitted filenames.
6. Add a Content-Disposition header to prevent browser rendering of sensitive file types.
7. Document the OWASP A05:2021 reference and real-world examples.

Prerequisites

- Python Flask and os.path module
- Basic HTTP and URL structure
- Python string methods
- Linux file system basics (for path traversal understanding)

Project 23: HTTPS & TLS Certificate Inspector ★ Beginner

Overview

Students build a Python tool that connects to any HTTPS website, extracts and displays its TLS certificate details (subject, issuer, validity dates, SANs, cipher suite, TLS version), and performs validation checks. The tool flags expired certificates, weak ciphers, and self-signed certificates. This builds practical awareness of how HTTPS works.

Main Steps & Procedures

1. Use Python's ssl module to connect to a target host and retrieve its certificate.
2. Parse and display all certificate fields: CN, SANs, issuer, validity dates, serial number.
3. Check if the certificate is expired or within 30 days of expiry.
4. Identify the TLS version and cipher suite negotiated.
5. Flag use of SHA-1 signatures or short RSA keys as weak.

Prerequisites

- Python ssl and socket modules
- Basic understanding of HTTPS and TLS
- Python datetime module
- No external libraries required

6. Test against a variety of popular websites and a self-signed certificate.
7. Generate a comparison table of certificate quality across 10 popular sites.

Project 24: API Security Testing Tool ★★★ Advanced

Overview

Students build a REST API with Flask that has common API security flaws (no authentication, mass assignment, excessive data exposure, lack of rate limiting), then write a Python testing harness that probes the API for these vulnerabilities. They fix each flaw and demonstrate the fix works against the same test harness.

Main Steps & Procedures

1. Build a Flask REST API with user and resource endpoints.
2. Introduce deliberate flaws: unauthenticated endpoints, mass assignment on user update, verbose error messages.
3. Write test cases that enumerate endpoints, attempt unauthorized access, and over-post fields.
4. Implement JWT-based authentication using PyJWT.
5. Add rate limiting using Flask-Limiter.
6. Fix mass assignment by whitelisting allowed fields in update endpoints.
7. Suppress verbose errors in production mode and return generic error messages.

Prerequisites

- Python Flask REST routing
- Python requests library
- PyJWT library (pip install pyjwt)
- Basic understanding of REST APIs and JSON

Category 4: Authentication & Access Control

Project 25: Multi-Factor Authentication (MFA) Simulator ★★ Intermediate

Overview

Students build a Flask web application that implements three MFA factors: something you know (password with bcrypt hashing), something you have (TOTP code via pyotp, compatible with Google Authenticator), and something you are (simulated biometric via a PIN pattern). This project teaches defense-in-depth and how real-world MFA works.

Main Steps & Procedures

1. Build a secure login page with bcrypt password hashing.
2. Integrate pyotp to generate a TOTP secret and produce a QR code for enrollment.
3. Implement TOTP verification: accept a 6-digit code valid within a 30-second window.
4. Add a simulated biometric step (pattern-based PIN as a placeholder).
5. Implement account lockout after 5 failed attempts.
6. Show session is only granted after all three factors pass.
7. Discuss real MFA bypass techniques (SIM swapping, SS7 attacks, phishing MFA codes).

Prerequisites

- Python Flask and sessions
- Python bcrypt and pyotp libraries
- Python qrcode library for QR generation
- Basic HTML forms

Project 26: Role-Based Access Control (RBAC) System ★★ Intermediate

Overview

Students design and implement an RBAC system in Python for a simulated hospital management portal. Users are assigned roles (Admin, Doctor, Nurse, Patient) and roles grant specific permissions on resources (read/write patient records, prescribe medication, view billing). Students demonstrate privilege escalation attacks against a broken implementation, then fix them.

Main Steps & Procedures

1. Model the permission matrix: roles, resources, and allowed operations.
2. Implement the role assignment system in SQLite.
3. Build Flask routes that enforce role checks before granting access.
4. Deliberately introduce a privilege escalation flaw (role stored in cookie).
5. Demonstrate the escalation: modify the cookie to gain admin access.
6. Fix by moving role to server-side session storage.
7. Add audit logging: record every access attempt with user, role, resource, and outcome.

Prerequisites

- Python Flask and sqlite3
- Python sessions and cookies
- Basic SQL (CREATE, INSERT, SELECT)
- Understanding of least-privilege principle

Project 27: Brute-Force Attack Simulator & Account Lockout Defense ★★ Intermediate

Overview

Students build a Flask login application without lockout protection, then write a Python brute-force script that hammers it with a dictionary of common passwords. They measure how quickly it

succeeds, then implement progressive lockout, CAPTCHA simulation, and IP-based rate limiting as defenses, measuring the effectiveness of each.

Main Steps & Procedures

1. Build a basic Flask login app with username/password stored in SQLite.
2. Write a Python brute-force client using requests and a 1,000-word password list.
3. Measure time to crack a common password (e.g., 'password123').
4. Implement account lockout after 5 attempts with a 15-minute timeout.
5. Add exponential backoff: each failed attempt increases response delay.
6. Implement IP-based rate limiting (max 10 requests/minute per IP).
7. Add a simple text-based CAPTCHA challenge and measure its effect on the brute-force script.

Prerequisites

- Python Flask and sqlite3
- Python requests library
- Python time and threading modules
- No external libraries required

Project 28: OAuth 2.0 Authorization Flow Simulator ★★★ Advanced

Overview

Students build a three-component OAuth 2.0 simulation in Python: an Authorization Server, a Resource Server, and a Client Application. They implement the Authorization Code flow, including state parameter (CSRF protection), authorization codes, access tokens, and refresh tokens. This demystifies how social logins (Google, GitHub) work under the hood.

Main Steps & Procedures

1. Study the OAuth 2.0 Authorization Code flow RFC 6749.
2. Build the Authorization Server: login, consent screen, code issuance.
3. Build the Resource Server: validate Bearer tokens and return protected resources.
4. Build the Client App: redirect user to Auth Server, exchange code for token, call Resource Server.
5. Implement the state parameter and show how it prevents CSRF in the OAuth flow.
6. Demonstrate an authorization code interception attack (missing PKCE).
7. Add PKCE (Proof Key for Code Exchange) and show the attack fails.

Prerequisites

- Python Flask
- Python secrets and hashlib modules
- Understanding of HTTP redirects and query parameters
- Basic understanding of tokens and sessions

Project 29: Password Manager (Secure Vault) ★★★ Advanced

Overview

Students build a CLI-based password manager in Python that stores credentials in an encrypted SQLite database. The master password is never stored; instead, it derives the encryption key via Argon2 (or PBKDF2). Credentials are encrypted with AES-256-GCM. Students also implement a breach-check feature using the HaveIBeenPwned API's k-anonymity model.

Main Steps & Procedures

1. Design the database schema: vault table (site, username, encrypted_password, iv).

Prerequisites

- Python cryptography library
- Python sqlite3 module

2. Implement master password key derivation using PBKDF2-HMAC-SHA256 with a stored salt.
3. Encrypt each credential entry with AES-256-GCM using the derived key.
4. Implement add, retrieve, update, and delete operations on the vault.
5. Implement the search function with partial-match support.
6. Integrate the HaveIBeenPwned API: hash the password, send first 5 chars, check response.
7. Add a clipboard-copy feature that clears the clipboard after 30 seconds.

- Python requests library (for HIBP API)
- Basic CLI argument parsing (argparse)

Project 30: JWT Authentication: Attacks & Defenses ★★★ Advanced

Overview

Students build a Flask API protected by JWT tokens, then exploit three well-known JWT vulnerabilities: the 'alg:none' attack, the algorithm confusion attack (RS256 to HS256), and weak secret brute-force. After demonstrating each exploit, they harden the API with strict algorithm pinning and strong secret key requirements.

Main Steps & Procedures

1. Build a Flask API with JWT-based authentication using PyJWT.
2. Implement the 'alg:none' attack: craft a token with no signature.
3. Implement the RS256-to-HS256 confusion attack using the public key as HMAC secret.
4. Brute-force a JWT signed with the weak secret 'password' using a wordlist.
5. Harden by explicitly specifying allowed_algorithms=['HS256'] in verify calls.
6. Enforce minimum secret key length (256 bits) and use secrets.token_hex(32).
7. Add token expiry, issuer claim validation, and an invalid token blocklist.

Prerequisites

- Python Flask
- PyJWT library
- Python base64 and json modules
- Python requests library

Project 31: Privilege Escalation Lab (Linux Permissions) ★★ Intermediate

Overview

Students set up a Linux VM with deliberate misconfigurations — SUID binaries, world-writable cron jobs, weak sudo rules — and write a Python enumeration script that discovers these vulnerabilities. They then exploit each finding to gain elevated privileges and write a hardening report with remediation scripts.

Main Steps & Procedures

1. Set up a Linux VM with intentional misconfigurations (SUID on /bin/vim, world-writable cron).
2. Write a Python script that enumerates: SUID/SGID binaries, world-writable files, sudo rules, cron jobs.
3. Exploit SUID binary to spawn a root shell.
4. Exploit world-writable cron job to execute arbitrary commands as root.

Prerequisites

- A Linux VM (Ubuntu/Kali)
- Python os, subprocess, and stat modules
- Basic Linux command line knowledge
- Understanding of file permissions and sudo

5. Exploit a misconfigured sudo rule (NOPASSWD on a shell command). 6. Write a Python hardening script that removes misconfigurations and fixes permissions. 7. Document each finding in a penetration testing report format.	

Category 5: Malware Analysis & Digital Forensics

Project 32: Malware Behavior Simulator (Benign Demonstration) ★★★ Advanced

Overview

Students simulate the behavior of common malware types in a completely isolated Python sandbox: a keylogger (logs keystrokes to a local file), a file enumerator (maps a directory tree like ransomware does before encrypting), and a simple persistence mechanism (writes a startup script). No network activity or real damage occurs. The goal is to understand malware behavior, not create real malware.

Main Steps & Procedures

1. Set up a completely isolated VM with no network access.
2. Implement a benign keylogger using Python pynput that logs to a local file only.
3. Implement a file enumerator that maps all documents in a test directory (for ransomware simulation).
4. Implement a persistence demonstration: write a startup entry to a test registry key or cron.
5. Build a monitoring script that detects all three behaviors using process and file-system watchers.
6. Document each behavior and map it to the MITRE ATT&CK framework.
7. Discuss legal and ethical boundaries: this code must never leave the isolated lab.

Prerequisites

- Isolated VM (mandatory — no network)
- Python pynput and watchdog libraries
- Python os and subprocess modules
- MITRE ATT&CK framework (reading only)

Project 33: File Metadata Forensics Investigator ★ Beginner

Overview

Students build a Python digital forensics tool that extracts and analyzes file metadata from images (EXIF: GPS, camera model, timestamp), PDF documents (author, creation tool, editing history), and Office files (last modified by, revision count). The tool generates a privacy risk report, demonstrating how metadata leaks sensitive information.

Main Steps & Procedures

1. Use Python's exifread library to extract EXIF data from JPEG/PNG images.
2. Parse GPS coordinates from EXIF and convert to human-readable decimal degrees.
3. Map GPS coordinates to a location using the Nominatim API (OpenStreetMap).
4. Use pypdf to extract metadata from PDF files.
5. Use python-docx to extract metadata from Word documents.
6. Flag high-risk fields: GPS data, author names, machine names, revision counts.
7. Generate a privacy report with a risk score for each file.

Prerequisites

- Python exifread and pypdf libraries
- Python python-docx library
- Python requests (for geocoding API)
- No security background required

Project 34: Disk Image File Recovery Tool ★★★ Advanced

Overview

Students implement a file carving tool in Python that recovers deleted files from a raw disk image by searching for known file signatures (magic bytes). The tool recovers JPEG, PNG, PDF, and ZIP files even without a filesystem. This introduces low-level forensics, binary analysis, and how 'deleted' data lingers on disk.

Main Steps & Procedures

1. Create a small test disk image containing known files using dd or a Python script.
2. Delete the files and simulate filesystem wiping.
3. Study magic byte signatures for JPEG (FF D8 FF), PNG (89 50 4E 47), PDF (%PDF), and ZIP (PK).
4. Write a binary scanner that slides across the raw image looking for these signatures.
5. Implement file boundary detection (JPEG end marker FFD9, PNG IEND chunk).
6. Extract and save each recovered file.
7. Verify recovered files open correctly and measure recovery success rate.

Prerequisites

- Python binary file I/O (rb mode)
- Python struct and bytes/bytearray knowledge
- Basic understanding of file systems
- Python os and hashlib modules

Project 35: Network Forensics: PCAP Timeline Reconstruction ★★★ Advanced

Overview

Students analyze a provided PCAP file of a simulated cyberattack using Python and Scapy, reconstructing a timeline of events: initial reconnaissance, exploitation attempt, data exfiltration, and C2 communication. The output is a structured incident report. This introduces network forensics methodology and incident response thinking.

Main Steps & Procedures

1. Obtain a sample attack PCAP file from a public dataset (e.g., CICIDS2017).
2. Write a Python script to extract all unique conversations (source IP, dest IP, port, protocol).
3. Identify the reconnaissance phase: high-volume SYN packets to many ports.
4. Identify the exploitation attempt: abnormal payload in HTTP or unusual protocol.
5. Detect data exfiltration: large outbound data transfers to a new external IP.
6. Reconstruct the attack timeline with timestamps.
7. Write a formal incident response report using a standard template (NIST IR framework).

Prerequisites

- Python Scapy library
- Public PCAP files (CICIDS, Netresec, PCAPR)
- Python pandas for timeline analysis
- Basic TCP/IP knowledge

Project 36: Ransomware Defense Simulator ★★★ Advanced

Overview

Students build a benign ransomware behavior simulation in a strictly isolated environment to understand how ransomware works: file enumeration, encryption (AES), and ransom note generation. They then build a honeypot-based defense system and a decryptor. The focus is entirely on defense and understanding — the tool is non-functional outside the isolated lab.

Main Steps & Procedures

1. Set up a completely isolated VM with a test directory of dummy files.

Prerequisites

- Isolated VM — mandatory, no network

2. Implement a file enumerator that lists all documents in the test directory.
3. Implement AES-256 file encryption on the dummy files.
4. Generate a ransom note text file.
5. Implement the decryptor using the stored key.
6. Build a honeypot defense: plant a canary file that triggers an alert if modified.
7. Implement a file-system watcher that detects mass encryption events and kills the simulated process.

- Python cryptography library
- Python watchdog library
- Python os and logging modules

Project 37: Memory Forensics: Process & String Extractor ★★★ Advanced

Overview

Students build a Python tool that analyzes a memory dump file (provided as a binary blob) to extract readable strings, identify running process artifacts, and search for patterns like email addresses, URLs, and IP addresses. This introduces memory forensics concepts and how investigators find evidence in RAM captures.

Main Steps & Procedures

1. Obtain a sample memory dump (Volatility Foundation provides test images).
2. Implement a string extractor: scan the binary for sequences of printable ASCII 4+ chars.
3. Search for patterns using regular expressions: email addresses, URLs, IP addresses.
4. Identify Windows PE headers (MZ signature) to locate loaded executables in memory.
5. Extract and display artifacts: process names, DLL names visible as strings.
6. Compare findings with the Volatility Framework output to validate.
7. Document the methodology and tools used in a forensic chain-of-custody report.

Prerequisites

- Python binary I/O and struct module
- Python re module for pattern matching
- Sample memory dump (from Volatility project)
- Basic understanding of OS process concepts

Project 38: Antivirus Engine Simulator (Signature-Based) ★★ Intermediate

Overview

Students build a simple signature-based antivirus scanner in Python that maintains a database of file hash signatures and YARA-style byte-pattern signatures. The scanner checks target files against the database and reports matches. Students add new signatures from public threat intelligence feeds and measure detection rates.

Main Steps & Procedures

1. Build a signature database: a JSON file containing MD5/SHA-256 hashes and byte patterns.
2. Implement hash-based scanning: compute hash of each scanned file and check against the database.
3. Implement pattern-based scanning: search file bytes for hex patterns from the signature database.
4. Source 20 harmless malware signatures from VirusTotal public reports or YARA rule repositories.
5. Create test files that trigger signatures to validate the scanner.
6. Implement recursive directory scanning.

Prerequisites

- Python hashlib and os modules
- Python binary file I/O
- Python JSON module
- Basic understanding of how antivirus works

7. Measure and report false positive rate, detection rate, and scan speed.	

Category 6: Privacy & Social Engineering

Project 39: Phishing Email Detector & Analyzer ★★ Intermediate

Overview

Students build a Python rule-based phishing detection engine that analyzes email headers, sender domains, URLs, and body text to generate a threat score. The tool flags mismatched Reply-To domains, lookalike URLs (paypa1.com vs paypal.com), urgency language, suspicious attachments, and SPF/DKIM header anomalies. A dataset of real phishing emails (PhishTank) is used for testing.

Main Steps & Procedures

1. Study email header structure and SPF/DKIM authentication fields.
2. Use Python's email module to parse raw .eml files.
3. Extract and validate SPF and DKIM header entries.
4. Implement homoglyph/lookalike domain detection (Levenshtein distance).
5. Build a keyword scorer for urgency language (verify, account suspended, click now).
6. Extract all URLs and flag those with mismatched display text vs. href.
7. Test against 50 phishing samples from PhishTank and measure detection accuracy.

Prerequisites

- Python email module (standard library)
- Python re module
- Python-Levenshtein library (optional)
- PhishTank public dataset (free to download)

Project 40: OSINT (Open Source Intelligence) Aggregator ★★ Intermediate

Overview

Students build a Python OSINT tool that aggregates publicly available information about a domain or organization: WHOIS data, DNS records, certificate transparency logs, Shodan-exposed services, and public GitHub repositories. The tool compiles a structured intelligence report. Students use only their own university domain as the target.

Main Steps & Procedures

1. Use python-whois to retrieve domain registration data.
2. Enumerate DNS records using dnspython (A, MX, NS, TXT, CNAME).
3. Query certificate transparency logs via crt.sh API to find subdomains.
4. Query the Shodan API for exposed services on discovered IPs.
5. Search public GitHub for the organization name to find accidentally committed secrets.
6. Compile all findings into a structured intelligence report.
7. Analyze the collected data to identify the organization's security footprint and risks.

Prerequisites

- Python requests and dnspython libraries
- Shodan API key (free tier available)
- python-whois library
- Only target your own organization's domain

Project 41: Social Engineering Awareness Training Tool ★ Beginner

Overview

Students build an interactive web-based security awareness quiz using Python Flask and JavaScript. The quiz presents realistic phishing email screenshots, fake login pages, and social engineering scenarios, asking users to identify the threat. The tool tracks scores, explains why each answer is correct or wrong, and generates a personalized learning path.

Main Steps & Procedures

1. Design 30+ scenario questions covering phishing, vishing, pretexting, and baiting.
2. Build a Flask backend that serves questions and evaluates answers.
3. Include realistic fake screenshots of phishing emails and spoofed login pages (created by the team).
4. Implement a scoring system with per-category breakdown.
5. Add detailed explanations for each question after it is answered.
6. Generate a personalized report highlighting the user's weakest areas.
7. Test the tool with at least 20 real users and analyze the results.

Prerequisites

- Python Flask
- Basic HTML/CSS and JavaScript
- Python JSON for question database
- No security background required

Project 42: Dark Web Data Breach Monitor ★★ Intermediate

Overview

Students build a Python privacy monitoring tool that checks whether a user's email or password has appeared in known data breaches using the HaveIBeenPwned API (k-anonymity model). The tool also monitors a list of email addresses and sends alerts when new breaches are detected. This teaches privacy, API integration, and k-anonymity.

Main Steps & Procedures

1. Study the k-anonymity API design used by HaveIBeenPwned.
2. Implement email breach checking: query HIBP email API and display breach details.
3. Implement password breach checking: SHA-1 hash the password, send first 5 chars, check suffix.
4. Build a watchlist: store monitored emails in a local SQLite database.
5. Implement a scheduler (schedule library) that checks the watchlist daily.
6. Send alert emails using Python's smtplib when a new breach is detected.
7. Display a detailed breach report: breach name, date, compromised data types.

Prerequisites

- Python requests and hashlib modules
- Python sqlite3 and smtplib modules
- Python schedule library
- HaveIBeenPwned API (free for personal use)

Project 43: Privacy Risk Assessor for Mobile Apps ★★ Intermediate

Overview

Students build a Python tool that parses Android APK manifests and iOS App Store permission declarations to identify excessive permission requests, tracking SDKs, and data collection practices. They then score each app on a privacy risk scale and compare practices across a set of popular apps.

Main Steps & Procedures

1. Use Python's zipfile module to unpack APK files and parse AndroidManifest.xml.
2. Extract all declared permissions and classify them by sensitivity level.

Prerequisites

- Python zipfile and xml.etree.ElementTree modules
- Python requests and BeautifulSoup
- Basic XML parsing knowledge

3. Cross-reference listed permissions with known tracking SDKs (Google Ads, Facebook, etc.).
4. Scrape App Store permission descriptions for a set of iOS apps using requests.
5. Implement a risk scoring algorithm based on permission count and sensitivity.
6. Analyze 10 popular apps and compare their privacy risk scores.
7. Produce a user-friendly privacy report explaining each permission in plain language.

- A set of APK files for analysis (downloadable legally)

Project 44: Deepfake Awareness Detector (Metadata Analysis) ★★ Intermediate

Overview

Students build a Python tool that analyzes images and videos for signs of AI generation or manipulation using metadata analysis and statistical pixel-level features (Error Level Analysis, compression artifacts). While full deepfake detection requires ML, this project focuses on the forensic signals that distinguish authentic media from manipulated media.

Main Steps & Procedures

1. Study EXIF metadata differences between camera photos and AI-generated images.
2. Implement Error Level Analysis (ELA): re-compress an image and compare with original.
3. Compute noise level uniformity across image regions (AI images tend to be too clean).
4. Check for inconsistent lighting by analyzing highlight and shadow distributions.
5. Extract and analyze metadata: AI-generated images often lack camera model data.
6. Build a scoring system that combines all signals into a manipulation likelihood score.
7. Test against a set of authentic photos and known AI-generated images (publicly available).

Prerequisites

- Python Pillow and numpy libraries
- Python scipy for statistical analysis
- Python exifread library
- Basic understanding of image compression

Project 45: Secure Communication Policy Analyzer ★ Beginner

Overview

Students build a Python tool that evaluates the security configuration of email and web domains: SPF, DKIM, DMARC, HTTPS enforcement, HSTS headers, and certificate quality. For a set of 20 local Egyptian organizations, they assess and score their email and web security posture, producing an actionable report with recommendations.

Main Steps & Procedures

1. Research SPF, DKIM, and DMARC record formats and their security roles.
2. Use dnspython to query SPF and DMARC TXT records for each domain.
3. Use Python's ssl module to check TLS version, cipher, and certificate quality.
4. Send an HTTP request and check for HSTS, X-Frame-Options, and CSP headers.
5. Build a weighted scoring system across all six security dimensions.

Prerequisites

- Python dnspython library
- Python ssl and requests modules
- Basic understanding of DNS and HTTP headers
- No external APIs required

6. Evaluate 20 Egyptian organization domains (universities, banks, government). 7. Present findings and recommendations in a formal report with remediation steps.	

Category 7: Secure Coding & DevSecOps

Project 46: Static Code Analyzer for Common Vulnerabilities ★★★ Advanced

Overview

Students build a Python static analysis tool that scans Python source code for common security vulnerabilities: hardcoded credentials, use of dangerous functions (eval, exec, pickle), SQL string concatenation, insecure random number generation, and command injection via subprocess with shell=True. The tool annotates the code with vulnerability warnings and severity ratings.

Main Steps & Procedures

1. Study the OWASP Top 10 categories and map them to Python-specific code patterns.
2. Use Python's ast module to parse source files into an Abstract Syntax Tree.
3. Implement detectors for hardcoded strings matching password/key/secret patterns.
4. Detect calls to dangerous functions: eval(), exec(), pickle.loads(), os.system().
5. Detect SQL string formatting: f-strings or % format in sqlite3.execute() calls.
6. Detect random.random() in security-sensitive contexts.
7. Generate an annotated report with file, line number, severity, and recommended fix.

Prerequisites

- Python ast module (standard library)
- Python re module
- Understanding of Python code structure
- Basic knowledge of common vulnerabilities

Project 47: Dependency Vulnerability Scanner ★★ Intermediate

Overview

Students build a Python tool that reads a project's requirements.txt, queries the OSV (Open Source Vulnerabilities) and PyPI APIs to find known CVEs in installed packages, and produces a prioritized remediation report. This introduces software supply chain security and the concept of vulnerable dependencies — a major real-world attack vector.

Main Steps & Procedures

1. Parse requirements.txt to extract package names and version constraints.
2. Query the OSV API (osv.dev) for known vulnerabilities in each package version.
3. Query the PyPI API to retrieve the latest stable version and changelog.
4. Cross-reference CVE severity (CVSS score) from the National Vulnerability Database.
5. Generate a prioritized report: critical vulnerabilities first, with upgrade paths.
6. Add a CI-friendly mode that exits with code 1 if critical CVEs are found.
7. Integrate with a sample GitHub Actions workflow to demonstrate DevSecOps automation.

Prerequisites

- Python requests library
- Basic JSON parsing
- Understanding of package versioning (semver)
- No external libraries required beyond requests

Project 48: Secure Code Review Checklist Automation ★★ Intermediate

Overview

Students build a Python tool that automates a security-focused code review checklist. Given a GitHub repository URL, the tool clones it, runs a suite of checks (insecure imports, hardcoded secrets using regex, missing input validation patterns, insecure configurations), and produces a pull-request-style annotated report.

Main Steps & Procedures

1. Use Python's subprocess to run git clone on a provided GitHub repository URL.
2. Walk the repository directory tree to find all Python files.
3. Run regex-based checks: hardcoded API keys, SSH private key patterns, AWS credentials.
4. Check for insecure imports: import hashlib usage of MD5, use of http vs https in URLs.
5. Detect missing input validation in Flask/Django views (no type checking on request data).
6. Generate a pull request style annotation report: file, line, finding, severity, recommendation.
7. Present the tool to review a real open-source project (with instructor approval).

Prerequisites

- Python subprocess and os modules
- Python re module
- git command line basics
- Python requests for GitHub API

Project 49: Container Security Scanner (Docker Image Auditor) ★★★ Advanced

Overview

Students build a Python tool that audits Dockerfiles and running container configurations for security misconfigurations: running as root, using latest tags, exposed sensitive ports, hardcoded secrets in ENV instructions, missing HEALTHCHECK, and outdated base images. The tool uses the Docker SDK for Python.

Main Steps & Procedures

1. Study Docker security best practices (CIS Docker Benchmark).
2. Use the Docker SDK for Python to list running containers and inspect their configuration.
3. Parse Dockerfiles to detect: FROM latest, USER root, ENV containing passwords.
4. Check for containers running with privileged mode or excessive capabilities.
5. Detect exposed ports that should not be public (e.g., database ports 3306, 5432).
6. Check for missing resource limits (CPU, memory).
7. Generate a security score for each container/image with prioritized remediation steps.

Prerequisites

- Docker Desktop installed
- Python docker library (pip install docker)
- Basic Docker knowledge (Dockerfile syntax)
- Basic understanding of Linux capabilities

Project 50: Secrets Detection Tool for Git Repositories ★★ Intermediate

Overview

Students build a Python pre-commit hook and repository scanner that detects accidentally committed secrets: API keys, passwords, private keys, tokens, and connection strings. The tool scans both current files and the entire Git history. This addresses one of the most common real-world security incidents.

Main Steps & Procedures

Prerequisites

1. Compile a library of regex patterns for common secret types (AWS keys, GitHub tokens, private keys).
2. Use Python subprocess to run git log and git show to iterate over commit history.
3. Scan each commit diff for pattern matches.
4. Implement entropy analysis: flag strings with high Shannon entropy as potential secrets.
5. Build a pre-commit hook that prevents a commit if secrets are detected.
6. Create a .gitignore and .gitattributes template for common sensitive files.
7. Test against a synthetic repository containing 10 different types of planted secrets.

- Python re and math modules
- Python subprocess for Git commands
- Basic understanding of Git commits and diffs
- Understanding of entropy (basic probability)

Project 51: Security Headers Hardening Tool ★ Beginner

Overview

Students build a Python Flask middleware that automatically adds all recommended HTTP security headers to every response (HSTS, CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy). They also build an audit tool that checks any website's headers and grades them using the securityheaders.com methodology.

Main Steps & Procedures

1. Research each security header: purpose, recommended value, and browser support.
2. Build a Flask “@after_request” middleware that injects all security headers.
3. Test each header with a browser developer tools demonstration.
4. Build a scanner that fetches headers from any URL using requests and grades them.
5. Implement a Content Security Policy builder: wizard-style interface for constructing CSP values.
6. Audit 10 popular websites and grade their header security (A+ to F).
7. Demonstrate a clickjacking attack prevented by X-Frame-Options.

Prerequisites

- Python Flask
- Python requests library
- Basic HTTP header knowledge
- Basic HTML and browser developer tools

Project 52: Input Validation & Sanitization Library ★★ Intermediate

Overview

Students design and implement a Python input validation library for web applications that validates and sanitizes user input across multiple data types: email addresses, phone numbers, URLs, file uploads, JSON payloads, and free-text fields. The library defends against XSS, command injection, path traversal, and ReDoS attacks.

Main Steps & Procedures

1. Define the library API: a Validator class with chainable rules.
2. Implement validators for: email (RFC 5322), phone number, URL (safe schemes only), filename.
3. Implement sanitizers for: HTML output (strip tags), SQL input (parameterize), shell input (shlex.quote).

Prerequisites

- Python re and html modules
- Python shlex module (standard library)
- Python mimetypes module
- Basic Flask for integration demo

4. Add a file upload validator: check MIME type, extension, and file size against allowlist.
5. Test each validator with a fuzzing suite of malicious inputs.
6. Demonstrate integration in a Flask application.
7. Write comprehensive documentation with security rationale for each rule.

Project 53: CI/CD Security Pipeline (DevSecOps Demo) ★★★ Advanced

Overview

Students build a complete DevSecOps pipeline using GitHub Actions that automatically runs security checks on every push: static analysis (bandit), dependency scanning, secret detection, container scanning (if applicable), and DAST testing against a locally deployed test app. They configure the pipeline to fail builds on critical findings.

Main Steps & Procedures

1. Create a sample vulnerable Flask application repository on GitHub.
2. Write a GitHub Actions workflow YAML that triggers on push.
3. Integrate Bandit (static analysis for Python) and configure severity thresholds.
4. Add safety check for dependency vulnerabilities.
5. Integrate a secret detection step using the Secrets Detection tool (Project 58).
6. Add a DAST step that runs a basic web scan (OWASP ZAP or nikto in a container).
7. Configure branch protection rules that require all security checks to pass before merge.

Prerequisites

- GitHub account and basic Git workflow
- YAML syntax basics
- Python bandit and safety packages
- Docker basics (for ZAP container step)

Category 8: IoT, Systems & Emerging Threats

Project 54: IoT Device Security Auditor ★★ Intermediate

Overview

Students build a Python tool that audits IoT device security without attacking real devices. The tool scans a local network (with permission) for IoT devices using mDNS and UPnP discovery, identifies device types, checks for default credential use via a known-default-credentials database, and tests for open Telnet/SSH on IoT-typical ports.

Main Steps & Procedures

1. Use Python's zeroconf library to discover mDNS-announced IoT devices on the local LAN.
2. Use the python-ssdp library to discover UPnP-enabled devices.
3. Identify device types from service names and vendor OUI lookups.
4. Check for open Telnet (port 23) and SSH (port 22) using socket scans.
5. Test for default credentials using a database of 200 common IoT default username/password pairs.
6. Generate a risk report per device with firmware update recommendations.
7. Discuss Mirai botnet and how default credential attacks compromised millions of IoT devices.

Prerequisites

- Python zeroconf and socket modules
- Network with instructor-approved IoT test devices
- Python csv for credentials database
- Only scan devices you own and have permission to test

Project 55: AI-Powered Intrusion Detection (ML-Based NIDS) ★★★ Advanced

Overview

Students train a machine learning classifier on the CICIDS2017 network traffic dataset to distinguish normal traffic from six attack types (DoS, port scan, brute force, web attack, infiltration, botnet). They compare Logistic Regression, Random Forest, and a simple Neural Network, then build a real-time classifier that evaluates incoming packets.

Main Steps & Procedures

1. Download and explore the CICIDS2017 dataset (CSV features, pre-extracted).
2. Perform data cleaning: handle missing values, encode categorical features.
3. Split into 80/20 train/test sets.
4. Train three classifiers: Logistic Regression, Random Forest, and MLP using scikit-learn.
5. Evaluate each with accuracy, precision, recall, F1 score, and confusion matrix.
6. Build a real-time simulation: feed new feature vectors from a PCAP and classify in real time.
7. Analyze false positive rates and discuss the challenge of deploying ML in IDS.

Prerequisites

- Python scikit-learn, pandas, and numpy
- CICIDS2017 dataset (free download)
- Basic understanding of ML classification
- Python matplotlib for visualization

Project 56: Smart Home Threat Model & Attack Simulator ★★ Intermediate

Overview

Students create a complete threat model for a smart home system (smart locks, cameras, thermostat, voice assistant) using the STRIDE methodology, then build a Python simulation of the smart home where they implement and demonstrate three specific attack scenarios: replay attacks on smart lock commands, insecure MQTT message interception, and firmware update spoofing.

Main Steps & Procedures

1. Map the smart home architecture: devices, communication channels, cloud APIs.
2. Apply STRIDE threat modeling: enumerate threats per component.
3. Build a Python MQTT client/broker simulation using the paho-mqtt library.
4. Demonstrate a replay attack: capture and replay an unlock command.
5. Demonstrate insecure MQTT: intercept unencrypted messages on the simulated broker.
6. Simulate firmware update spoofing: replace a firmware image with a tampered one.
7. Implement fixes for each attack and verify they work.

Prerequisites

- Python paho-mqtt library
- STRIDE threat modeling methodology (study material)
- Python socket module
- mosquitto MQTT broker (free, for local simulation)

Project 57: Cloud Security Misconfiguration Detector (AWS S3) ★★★ Advanced

Overview

Students build a Python auditing tool that checks AWS S3 bucket configurations for common misconfigurations: public read/write access, missing server-side encryption, disabled versioning, missing access logging, and overly permissive bucket policies. They test against their own AWS free-tier buckets with deliberately introduced misconfigurations.

Main Steps & Procedures

1. Set up an AWS free-tier account and create test S3 buckets with known misconfigurations.
2. Use the boto3 library to programmatically retrieve bucket ACLs, policies, and settings.
3. Check for public access: ACL AllUsers, bucket policy with Principal: *.
4. Check for missing SSE encryption (AES-256 or KMS).
5. Check for disabled versioning and missing MFA Delete.
6. Check for missing access logging.
7. Generate a CIS AWS Foundations Benchmark-aligned report with pass/fail for each check.

Prerequisites

- AWS free-tier account
- Python boto3 library
- AWS IAM basic concepts
- Python JSON for policy parsing

Project 58: Adversarial Machine Learning: Fooling a Classifier ★★★ Advanced

Overview

Students train a simple image classifier (MNIST digit recognition) and then craft adversarial examples using the FGSM (Fast Gradient Sign Method) to cause misclassification. They then implement adversarial training and input preprocessing defenses and measure their effectiveness. This introduces the security risks inherent in AI/ML systems.

Main Steps & Procedures

1. Train a convolutional neural network on the MNIST dataset using PyTorch or TensorFlow.
2. Achieve >99% accuracy on the clean test set.

Prerequisites

- Python PyTorch or TensorFlow
- Python numpy and matplotlib

3. Implement the FGSM attack: compute gradient of loss w.r.t. input, add scaled perturbation.
4. Measure how accuracy drops under the attack at different epsilon (perturbation strength) values.
5. Visualize adversarial examples: show the original vs. perturbed image and misclassification.
6. Implement adversarial training: include adversarial examples in the training set.
7. Re-evaluate and compare robustness before and after adversarial training.

- Basic understanding of neural networks
- MNIST dataset (auto-downloaded by PyTorch/TF)

Project 59: Zero-Trust Architecture Simulator ★★★ Advanced

Overview

Students build a Python simulation of a Zero-Trust network architecture: every resource access requires authentication and authorization, no implicit trust is given based on network location, and all access is logged. They model a corporate network with multiple user roles and resource types, implement continuous verification, and compare it with a traditional perimeter-based model.

Main Steps & Procedures

1. Study Zero-Trust principles: verify explicitly, least privilege, assume breach.
2. Model the network: users, devices, resources, and the policy enforcement point.
3. Implement a policy engine that evaluates every access request: identity + device health + resource.
4. Build a device health check simulator: is the device patched? Is MFA enabled?
5. Implement micro-segmentation: users can only access explicitly permitted resources.
6. Simulate a compromised internal device and show it cannot move laterally.
7. Compare with a traditional perimeter model and demonstrate the lateral movement risk.

Prerequisites

- Python Flask (for policy engine API)
- Python sqlite3 for access log
- Understanding of network security concepts
- Python dataclasses or OOP

Project 60: Cybersecurity CTF (Capture the Flag) Challenge Creator ★★ Intermediate

Overview

Students build a self-contained web-based Capture the Flag platform in Python Flask. They design and implement 10 challenges across multiple categories (crypto, web, forensics, misc), hide flags in various forms (encrypted files, steganographic images, vulnerable web pages), and present the platform to peers in a live CTF competition.

Main Steps & Procedures

1. Design 10 challenges: 3 crypto, 3 web, 2 forensics, 2 misc, each with a hidden flag.
2. Build the CTF platform: Flask backend with challenge listings, submission form, and scoreboard.
3. Implement flag submission and verification with per-team tracking.
4. Deploy challenge servers: a vulnerable web page, an encoded file, a stego image.

Prerequisites

- Python Flask and sqlite3
- Knowledge from multiple previous projects (crypto, web, forensics)
- Basic HTML/CSS for the platform UI
- Python hashlib for flag hashing

5. Add hints system: each hint deducts points from the team's score. 6. Build a live scoreboard that updates every 60 seconds. 7. Host a live 2-hour CTF competition for classmates.	

#	Project Title	Category	Difficulty	Key Skill Focus
1	Classical Cipher Toolkit	Cryptography	★ Beginner	Python strings/crypto
2	File Encryption & Decryption Tool	Cryptography	★★ Intermediate	AES encryption
3	Password Strength Analyzer & Secure Generator	Cryptography	★ Beginner	secrets module
4	Hash Function Explorer & Password Cracker	Cryptography	★★ Intermediate	hashlib
5	Steganography Tool: Hiding Data in Images	Cryptography	★★ Intermediate	Pillow/LSB
6	Digital Signature Simulator	Cryptography	★★ Intermediate	RSA cryptography
7	Secure Messaging App (End-to-End Encryption Simulation)	Cryptography	★★★ Advanced	socket/AES
8	Blockchain Mini-Ledger	Cryptography	★★★ Advanced	hashlib/OOP
9	Port Scanner & Service Fingerprinter	Network Sec.	★★ Intermediate	socket/threading
10	Network Packet Sniffer & Analyzer	Network Sec.	★★★ Advanced	Scapy
11	HTTP Traffic Log Analyzer	Network Sec.	★ Beginner	file I/O/regex
12	DNS Enumeration & Spoofing Demo	Network Sec.	★★ Intermediate	dnspython/Scapy
13	Firewall Rule Simulator	Network Sec.	★★ Intermediate	JSON/socket
14	VPN Concept Demonstrator (TLS Tunnel Simulation)	Network Sec.	★★★ Advanced	ssl module
15	Wi-Fi Network Security Auditor (Passive)	Network Sec.	★★ Intermediate	Scapy/PCAP
16	Network Intrusion Detection System (Rule-Based)	Network Sec.	★★★ Advanced	Scapy/JSON rules
17	SQL Injection Exploit & Defense Lab	Web Security	★★ Intermediate	Flask/sqlite3
18	Cross-Site Scripting (XSS) Attack & Patch	Web Security	★★ Intermediate	Flask/Jinja2
19	CSRF Attack Demonstrator & Token Defense	Web Security	★★ Intermediate	Flask/sessions
20	Web Vulnerability Scanner (Lightweight)	Web Security	★★★ Advanced	requests/bs4
21	Broken Authentication: Session Hijacking Lab	Web Security	★★★ Advanced	Flask/secrets
22	Directory Traversal & File Inclusion Attack Lab	Web Security	★★ Intermediate	Flask/os.path
23	HTTPS & TLS Certificate Inspector	Web Security	★ Beginner	ssl/socket
24	API Security Testing Tool	Web Security	★★★ Advanced	Flask/PyJWT
25	Multi-Factor Authentication (MFA) Simulator	Auth & Access	★★ Intermediate	Flask/bcrypt/pyotp

#	Project Title	Category	Difficulty	Key Skill Focus
26	Role-Based Access Control (RBAC) System	Auth & Access	★★ Intermediate	Flask/sqlite3
27	Brute-Force Attack Simulator & Account Lockout Defense	Auth & Access	★★ Intermediate	requests/Flask
28	OAuth 2.0 Authorization Flow Simulator	Auth & Access	★★★ Advanced	Flask/OAuth
29	Password Manager (Secure Vault)	Auth & Access	★★★ Advanced	cryptography lib
30	JWT Authentication: Attacks & Defenses	Auth & Access	★★★ Advanced	PyJWT/Flask
31	Privilege Escalation Lab (Linux Permissions)	Auth & Access	★★ Intermediate	os/subprocess
32	Malware Behavior Simulator (Benign Demonstration)	Malware & Forensics	★★★ Advanced	pynput/watchdog
33	File Metadata Forensics Investigator	Malware & Forensics	★ Beginner	exifread/pypdf
34	Disk Image File Recovery Tool	Malware & Forensics	★★★ Advanced	binary I/O
35	Network Forensics: PCAP Timeline Reconstruction	Malware & Forensics	★★★ Advanced	Scapy/pandas
36	Ransomware Defense Simulator	Malware & Forensics	★★★ Advanced	cryptography/watchdog
37	Memory Forensics: Process & String Extractor	Malware & Forensics	★★★ Advanced	binary I/O/struct
38	Antivirus Engine Simulator (Signature-Based)	Malware & Forensics	★★ Intermediate	hashlib/os
39	Phishing Email Detector & Analyzer	Privacy & Social Eng.	★★ Intermediate	email/re/Levenshtein
40	OSINT (Open Source Intelligence) Aggregator	Privacy & Social Eng.	★★ Intermediate	requests/dnspython
41	Social Engineering Awareness Training Tool	Privacy & Social Eng.	★ Beginner	Flask/JavaScript
42	Dark Web Data Breach Monitor	Privacy & Social Eng.	★★ Intermediate	requests/hashlib
43	Privacy Risk Assessor for Mobile Apps	Privacy & Social Eng.	★★ Intermediate	zipfile/XML
44	Deepfake Awareness Detector (Metadata Analysis)	Privacy & Social Eng.	★★ Intermediate	Pillow/numpy
45	Secure Communication Policy Analyzer	Privacy & Social Eng.	★ Beginner	dnspython/ssl
46	Static Code Analyzer for Common Vulnerabilities	Secure Coding	★★★ Advanced	ast module
47	Dependency Vulnerability Scanner	Secure Coding	★★ Intermediate	requests/JSON
48	Secure Code Review Checklist Automation	Secure Coding	★★ Intermediate	subprocess/re
49	Container Security Scanner (Docker Image Auditor)	Secure Coding	★★★ Advanced	docker SDK

#	Project Title	Category	Difficulty	Key Skill Focus
50	Secrets Detection Tool for Git Repositories	Secure Coding	★★ Intermediate	re/subprocess
51	Security Headers Hardening Tool	Secure Coding	★ Beginner	Flask/requests
52	Input Validation & Sanitization Library	Secure Coding	★★ Intermediate	re/shlex
53	CI/CD Security Pipeline (DevSecOps Demo)	Secure Coding	★★★ Advanced	GitHub Actions/YAML
54	IoT Device Security Auditor	IoT & Systems	★★ Intermediate	zeroconf/socket
55	AI-Powered Intrusion Detection (ML-Based NIDS)	IoT & Systems	★★★ Advanced	scikit-learn/pandas
56	Smart Home Threat Model & Attack Simulator	IoT & Systems	★★ Intermediate	paho-mqtt
57	Cloud Security Misconfiguration Detector (AWS S3)	IoT & Systems	★★★ Advanced	boto3
58	Adversarial Machine Learning: Fooling a Classifier	IoT & Systems	★★★ Advanced	PyTorch
59	Zero-Trust Architecture Simulator	IoT & Systems	★★★ Advanced	Flask/sqlite3
60	Cybersecurity CTF (Capture the Flag) Challenge Creator	IoT & Systems	★★ Intermediate	Flask/sqlite3